

LOW-LEVEL DESIGN

User Posts & Albums Viewer

Client-side presentation-layer web application

Field	Value
Document Version	v0.3
Status	Draft - requirements clarification incorporated
Prepared By	Atul Sharma
Technology	HTML5, CSS3, JavaScript (ES6+), jQuery 4.0.0
Data Source	JSONPlaceholder REST API

Revision Summary

Requirements clarification incorporated: Implementation details updated for independent Posts/Albums requests, simultaneous panels, complete API URLs, expected results, and explicit name handling.

1. Purpose

This LLD translates the updated requirements and HLD into implementation-level design for the client-side presentation layer. It defines project structure, exact API calls, expected response shapes, state, DOM structure, request orchestration, independent loading/error handling, rendering, and stale-response protection.

2. Requirements Baseline

ID	Requirement	LLD response
FR-01	Retrieve users	getUsers() + loadUsers()
FR-02	Display first-name + last-name value	renderUsers() uses API user.name as the display value
FR-03	Select user	Delegated click handling + handleUserSelection()
FR-04	Retrieve Posts	getUserPosts(userId)
FR-05	Retrieve Albums	getUserAlbums(userId)
FR-06	Posts and Albums visible	Separate #posts-section and #albums-

ID	Requirement	LLD response
	simultaneously	section are rendered together
FR-07	Loading feedback	Independent user/post/album loading messages
FR-08	Error handling	Independent users/posts/albums error states
FR-09	Dynamic update	jQuery DOM updates without page refresh

3. Scope and Design Boundaries

3.1 In Scope

- Presentation-layer HTML/CSS/JS/jQuery implementation.
- Initial user retrieval and rendering.
- User selection.
- Separate Posts and Albums API requests.
- Simultaneous Posts and Albums presentation.
- Independent loading/error/empty states.
- Client-side stale-response protection.

3.2 Out of Scope

- Custom backend/business/service layer.
- Database or persistence.
- Authentication/authorization.
- Registration/login.
- Application-owned CRUD.

4. Proposed Project Structure

```

user-posts-albums-viewer/
|
+- index.html
+- css/
|  +- style.css
+- js/
|  +- api.js
|  +- ui.js
|  +- app.js
+- docs/
|  +- RUD_User_Posts_Albums_v1.2.pdf
|  +- SRS_User_Posts_Albums_v0.2.pdf
|  +- HLD_User_Posts_Albums_v0.3.pdf
|  +- LLD_User_Posts_Albums_v0.3.pdf
+- README.md

```

File	Responsibility
index.html	Stable page structure and UI containers

File	Responsibility
css/style.css	Layout, responsive behavior, selected state, loading/error/empty styling
js/api.js	JSONPlaceholder request functions only
js/ui.js	DOM rendering and UI state updates only
js/app.js	State, event handling, request orchestration, and coordination
README.md	Setup, usage, and project overview

5. Runtime Flow

1. On document ready, loadUsers() starts the initial request.
2. The client requests GET https://jsonplaceholder.typicode.com/users.
3. Returned users are stored in memory and rendered using user.name.
4. The user selects a user control.
5. The selected user is stored and old detail content/state is cleared.
6. Two independent requests start for the selected user: Posts and Albums.
7. The requests may execute concurrently; each request has its own loading/error lifecycle.
8. When a response succeeds, its dataset is stored and its panel is rendered.
9. Both Posts and Albums panels remain visible at the same time.
10. A request ID prevents a response belonging to an older user selection from changing the current UI.

6. Client-Side State Model

```
const state = {  
  users: [],  
  selectedUser: null,  
  posts: [],  
  albums: [],  
  usersLoading: false,  
  postsLoading: false,  
  albumsLoading: false,  
  usersError: null,  
  postsError: null,  
  albumsError: null,  
  detailRequestId: 0  
};
```

State field	Type	Responsibility
users	Array	Retrieved user records
selectedUser	Object / null	Current selected user
posts	Array	Posts for selected user
albums	Array	Albums for selected user

State field	Type	Responsibility
usersLoading	Boolean	Users request state
postsLoading	Boolean	Posts request state
albumsLoading	Boolean	Albums request state
usersError	String / null	Initial user request error
postsError	String / null	Posts request error
albumsError	String / null	Albums request error
detailRequestId	Number	Prevents stale selected-user responses from updating current UI

7. HTML / DOM Structure

```

<body>
  <header>
    <h1>User Posts & Albums Viewer</h1>
  </header>

  <main>
    <section id="users-panel">
      <h2>Users</h2>
      <div id="users-loading" hidden></div>
      <div id="users-error" hidden></div>
      <div id="users-list"></div>
    </section>

    <section id="details-panel">
      <h2 id="selected-user"></h2>

      <div id="posts-section">
        <h3>Posts</h3>
        <div id="posts-loading" hidden></div>
        <div id="posts-error" hidden></div>
        <div id="posts-content"></div>
      </div>

      <div id="albums-section">
        <h3>Albums</h3>
        <div id="albums-loading" hidden></div>
        <div id="albums-error" hidden></div>
        <div id="albums-content"></div>
      </div>
    </section>
  </main>
</body>

```

Element	Responsibility
#users-list	Render selectable users
#users-loading	Show initial users loading state
#users-error	Show initial users error state
#selected-user	Show selected user display name

Element	Responsibility
#posts-section	Stable Posts panel
#posts-loading	Posts request loading state
#posts-error	Posts request error state
#posts-content	Render Posts dataset
#albums-section	Stable Albums panel
#albums-loading	Albums request loading state
#albums-error	Albums request error state
#albums-content	Render Albums dataset

UI rule: Posts and Albums are separate panels in the same details area and are rendered simultaneously. There is no Posts/Albums tab switch and no shared content area that hides one dataset.

8. API Layer (api.js)

api.js contains transport logic only and does not modify the DOM.

8.1 Base URL

```
const API_BASE_URL = "https://jsonplaceholder.typicode.com";
```

8.2 Retrieve Users

Complete URL: <https://jsonplaceholder.typicode.com/users>

```
function getUsers() {
  return $.ajax({
    url: `${API_BASE_URL}/users`,
    method: "GET",
    dataType: "json"
  });
}
```

8.3 Retrieve Posts

Complete URL pattern: <https://jsonplaceholder.typicode.com/users/{userId}/posts>

Example for userId = 1: <https://jsonplaceholder.typicode.com/users/1/posts>

```
function getUserPosts(userId) {
  return $.ajax({
    url: `${API_BASE_URL}/users/${userId}/posts`,
    method: "GET",
    dataType: "json"
  });
}
```

8.4 Retrieve Albums

Complete URL pattern: `https://jsonplaceholder.typicode.com/users/{userId}/albums`

Example for `userId = 1`: `https://jsonplaceholder.typicode.com/users/1/albums`

```
function getUserAlbums(userId) {  
  return $.ajax({  
    url: `${API_BASE_URL}/users/${userId}/albums`,  
    method: "GET",  
    dataType: "json"  
  });  
}
```

8.5 Request Independence

Posts and Albums are separate API resources and separate HTTP requests. They should not be represented as one combined endpoint. They may be initiated concurrently.

9. Expected API Results

9.1 Users Response

Endpoint: GET `https://jsonplaceholder.typicode.com/users`

```
[  
  {  
    "id": 1,  
    "name": "Leanne Graham",  
    "username": "Bret",  
    "email": "Sincere@april.biz"  
  }  
]
```

9.2 Posts Response

Endpoint: GET `https://jsonplaceholder.typicode.com/users/1/posts`

```
[  
  {  
    "userId": 1,  
    "id": 1,  
    "title": "...",  
    "body": "..."  
  }  
]
```

9.3 Albums Response

Endpoint: GET `https://jsonplaceholder.typicode.com/users/1/albums`

```
[  
  {  
    "userId": 1,  
    "id": 1,
```

```
"title": "..."  
}  
]
```

10. Name Handling

JSONPlaceholder supplies one name property rather than separate firstName and lastName properties. Therefore the implementation uses user.name as the first-name + last-name display value.

API response: name = "Leanne Graham"
UI display: Leanne Graham

Name rule: No custom backend transformation and no fragile string-splitting rule is required by the current requirements.

11. Application Logic (app.js)

11.1 Initialization

```
$(function () {  
  loadUsers();  
  bindEvents();  
});
```

11.2 Initial User Retrieval

```
function loadUsers() {  
  state.usersLoading = true;  
  state.usersError = null;  
  showUsersLoading();  
  clearUsersError();  
  
  return getUsers()  
    .done(function (users) {  
      state.users = users;  
      renderUsers(state.users);  
    })  
    .fail(function () {  
      state.usersError = "Failed to fetch users.";  
      showUsersError(state.usersError);  
    })  
    .always(function () {  
      state.usersLoading = false;  
      hideUsersLoading();  
    });  
}
```

11.3 User Selection

```
function handleUserSelection(user) {  
  if (!user || !user.id) {  
    showDetailError("Unable to identify the selected user.");  
    return;  
  }  
}
```

```

}

state.selectedUser = user;
state.posts = [];
state.albums = [];
state.postsError = null;
state.albumsError = null;
state.detailRequestId += 1;

$("#selected-user").text(user.name);
clearPostsContent();
clearAlbumsContent();
clearPostsError();
clearAlbumsError();

loadUserDetails(user.id, state.detailRequestId);
}

```

11.4 Independent Posts and Albums Loading

```

function loadUserDetails(userId, requestId) {
  state.postsLoading = true;
  state.albumsLoading = true;
  showPostsLoading();
  showAlbumsLoading();

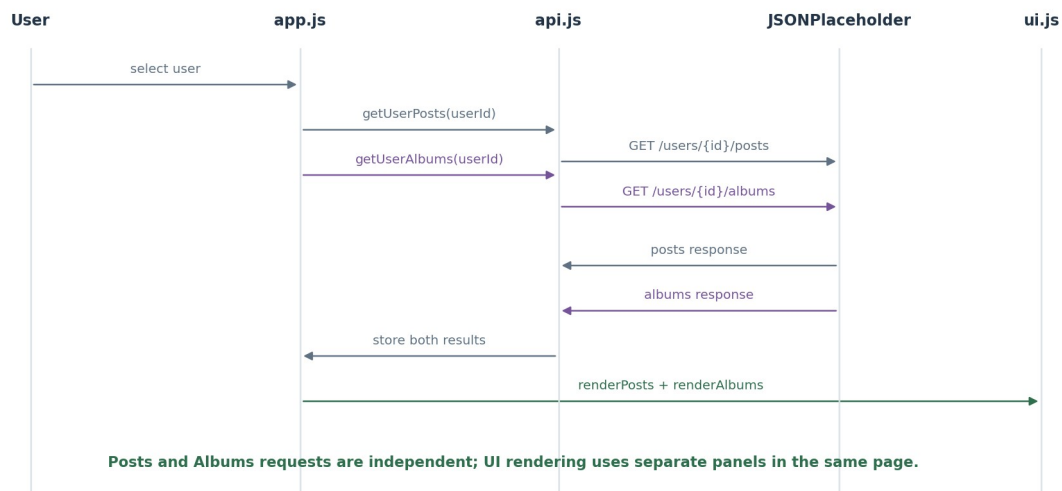
  return $.when(
    getUserPosts(userId),
    getUserAlbums(userId)
  )
  .done(function (posts, albums) {
    if (requestId !== state.detailRequestId) return;

    state.posts = posts[0];
    state.albums = albums[0];
    renderPosts(state.posts);
    renderAlbums(state.albums);
  })
  .fail(function () {
    if (requestId !== state.detailRequestId) return;
    showDetailError("Failed to load user Posts and/or Albums.");
  })
  .always(function () {
    if (requestId !== state.detailRequestId) return;
    state.postsLoading = false;
    state.albumsLoading = false;
    hidePostsLoading();
    hideAlbumsLoading();
  });
}

```

Important: \$.when() coordinates two separate AJAX operations. It does not convert Posts and Albums into one API request.

Figure 1. Independent Request Sequence



12. UI Rendering (ui.js)

12.1 Render Users

```

function renderUsers(users) {
  const $list = $("#users-list");
  $list.empty();
  $.each(users, function (_, user) {
    const $button = $("", {
      type: "button",
      class: "user-item",
      text: user.name,
      "data-user-id": user.id
    });
    $list.append($button);
  });
}

```

12.2 Render Posts

```

function renderPosts(posts) {
  const $content = $("#posts-content");
  $content.empty();
  if (posts.length === 0) {
    $content.append($("<p>").text("No posts available for this user."));
    return;
  }
  const $list = $("<ul>");
  $.each(posts, function (_, post) {
    const $item = $("<li>");
    $("<strong>").text(post.title).appendTo($item);
    $("<p>").text(post.body).appendTo($item);
    $item.appendTo($list);
  });
  $content.append($list);
}

```

12.3 Render Albums

```
function renderAlbums(albums) {
  const $content = $("#albums-content");
  $content.empty();
  if (albums.length === 0) {
    $content.append("<p>").text("No albums available for this user.");
    return;
  }
  const $list = "<ul>";
  $.each(albums, function (_, album) {
    "<li>".text(album.title).appendTo($list);
  });
  $content.append($list);
}
```

13. Event Handling

Because user controls are created dynamically, delegated event handling is used on #users-list. There is no Posts/Albums tab event because both sections remain visible simultaneously.

```
function bindEvents() {
  $("#users-list").on("click", ".user-item", function () {
    const userId = Number($(this).data("user-id"));
    const user = state.users.find(function (item) {
      return item.id === userId;
    });
    handleUserSelection(user);
  });
}
```

14. Loading, Error, and Empty-State Handling

- Users has its own loading/error state.
- Posts and Albums are separately represented in the DOM and can expose their own loading/error state.
- Empty arrays produce explicit empty messages.
- Selecting another user clears old detail data before new requests begin.
- A request ID prevents stale responses from older selections from updating the current UI.

15. Data Shapes Used by the UI

15.1 User

```
{
  "id": 1,
  "name": "Leanne Graham"
}
```

Used fields: id, name.

15.2 Post

```
{
  "userId": 1,
  "id": 1,
  "title": "...",
}
```

```
"body": "..."  
}
```

Used fields: title, body.

15.3 Album

```
{  
  "userId": 1,  
  "id": 1,  
  "title": "..."  
}
```

Used fields: title.

16. Concurrency and Request Coordination

Posts and Albums are independent HTTP resources. \$.when() can be used to coordinate completion, but it does not turn them into one API call. Each function sends its own GET request.

17. Security Considerations

- API-provided text is treated as untrusted input.
- jQuery .text() is preferred over raw HTML injection for external values.
- No API credentials or authentication secrets are required.
- No authentication or authorization layer is included.

18. Maintainability and Design Rules

- Keep API transport in api.js.
- Keep DOM rendering in ui.js.
- Keep state/workflow orchestration in app.js.
- Use stable IDs/classes for DOM targeting.
- Use the selected user ID to construct endpoint URLs.
- Keep Posts and Albums as separate datasets and separate UI panels.
- Avoid unnecessary abstraction because the scope remains small.

19. Requirement Traceability

Requirement	LLD implementation
Retrieve users	getUsers() + loadUsers()
Display first + last name	renderUsers() using API user.name
Select user	Delegated click handler + handleUserSelection()
Retrieve Posts	getUserPosts(userId)
Retrieve Albums	getUserAlbums(userId)
Simultaneous Posts + Albums	#posts-section + #albums-section rendered together
Loading	Separate user/posts/albums loading states
Errors	Separate user/posts/albums error handling
Dynamic update	jQuery DOM updates without full-page refresh

20. Implementation Checklist

- jQuery 4.0.0 is loaded.
- GET /users works on document ready.
- Users display the API name value.
- User controls are selectable.
- Selected user name is shown.
- Posts and Albums use the selected user ID.
- Posts and Albums are separate API requests.
- Posts and Albums can run concurrently.
- Posts and Albums are both visible after selection.
- Loading feedback is visible during retrieval.
- Errors are shown in the appropriate area.
- Empty arrays produce meaningful empty states.
- Selecting another user replaces previous detail data.
- Stale responses from older selections are ignored.
- No full-page refresh is required.
- API-provided text is rendered safely.

21. Status

LLD v0.3 - Draft. Updated to align with the latest requirements clarification and HLD: presentation-layer-only implementation, separate Posts/Albums requests, complete URLs and expected results, explicit name handling, and simultaneous Posts + Albums rendering.